



北京科技大学天津学院

Tianjin College, University of Science and Technology Beijing

《嵌入式人工智能基础与应用》 结课报告

题目：《基于 K230D 的桌面学习状态 AI 视觉识别》

班级：人工智能 2305

组员：杨再裕 何梓扬 吕中天

学号：235160541 235160513 235160503

2026 年 06 月

一、项目简介

本项目用 K230D BOX 开发板为核心，使用板载摄像头实时采集桌面学习场景画面，通过 YOLOv8n 目标检测模型识别画面中的人、手机和书本，结合基于时间窗口去抖的有限状态机，自动判定学习者的学习状态（玩手机/看书/检测到人），并在 LCD 上叠加显示检测结果，最后通过 12Pin GPIO 排针驱动的三色 LED 模块给出视觉反馈。系统以 25 FPS 的速度在端侧独立运行，不依赖任何云端推理，模型经过 INT8 量化后体积约 3.2MB，可以全部装入开发板。本项目完整跑通了 "数据集准备 → 模型训练 → 量化转换 → 端侧部署 → 状态判定 → 外设响应" 的端到端流程。

二、模型训练与部署

2.1 数据集

项目	数值
来源	https://github.com/Whiffe/SCB-dataset
原始图片	5 015 张（3 970 train / 1 024 val）
标注格式	YOLO `.txt`（归一化中心点 + 宽高）
类别数	3（person / phone / book）
标注框总数	25 810

类别分布（训练集 + 验证集合计）：

类别	标注框数	占比
person	11 207	43.4%
phone	10 841	42.0%
book	3 762	14.6%

数据预处理：

- 92% 图片原本为 RGBA 模式，统一转为 RGB + JPG
- 4 个标注文件中存在坐标越界/零宽高错误，已修复
- 21 个孤立标注（找不到对应图片）已剔除

2.2 模型训练

使用 Ultralytics YOLOv8n 作为基础模型，输入分辨率 320×320。

```
yolo detect train \
  data=dataset_study_yolo/dataset.yaml \
  model=yolov8n.pt \
  epochs=100 \
  imgsz=320 \
  batch=4 \
```

```
workers=0 \
patience=20 \
cos_lr=True \
close_mosaic=10 \
project=runs_study \
name=yolov8n_study_320
```

训练环境： CPU (i9-12900H), 约 3.8 it/s, 每 epoch 约 4.3 分钟。

关键超参：

参数	值	说明
model	yolov8n.pt	预训练权重
imgsz	320	与部署一致
batch	4	CPU 训练限制
optimizer	AdamW (auto)	Ultralytics 自动选择
lr0	0.001429	AdamW 自动
cos_lr	True	余弦学习率衰减
close_mosaic	10	最后 10 轮关闭 mosaic

2.3 训练结果

Epoch	mAP50	mAP50-95
1	0.265	0.137
20	0.478	0.292
40	0.542	0.351
60	0.572	0.374
80	0.599	0.399
90 (最佳)	**0.610**	**0.408**
98	0.608	0.404

最终 mAP50 = 0.610, mAP50-95 = 0.408。受 320×320 小输入尺寸 + CPU 训练 + 3 类限制, 未达到 YOLOv8 通用 0.80 目标, 但足以在桌面场景下完成实用检测。

2.4 模型转换

ONNX 导出：

```
yolo export model=best.pt format=onnx imgsz=320 opset=12 simplify=True
```

输出 best.onnx, 11.6 MB, 简化图, 输入 (1, 3, 320, 320) BCHW。

KModel 转换 (nncase 2.10.0)：

```
# 1) 加载 ONNX
import nncase
compile_options = nncase.CompileOptions()
compile_options.target = "k230"
compile_options.preprocess = True
compile_options.input_type = "uint8"
compile_options.input_shape = [1, 3, 320, 320]
compile_options.input_layout = "NCHW"
```

```

compile_options.mean = [0, 0, 0]
compile_options.std = [255, 255, 255]

compiler = nncase.Compiler(compile_options)
with open("best.onnx", "rb") as f:
    compiler.import_onnx(f.read())

# 2) PTQ 量化校准 (100 张代表性图片)
ptq_options = nncase.PTQTensorOptions()
ptq_options.samples_count = 100
ptq_options.calibrate_method = "Kld"
ptq_options.quant_type = "uint8"
ptq_options.w_quant_type = "uint8"
ptq_options.set_tensor_data(calib_data)
compiler.use_ptq(ptq_options)

# 3) 编译 + 保存
compiler.compile()
with open("study_yolov8n_320.kmodel", "wb") as f:
    f.write(compiler.gencode_tobytes())

```

输出 study_yolov8n_320.kmodel, **3.2 MB** (int8 量化后压缩到原 ONNX 的 28%)。

部署路径：

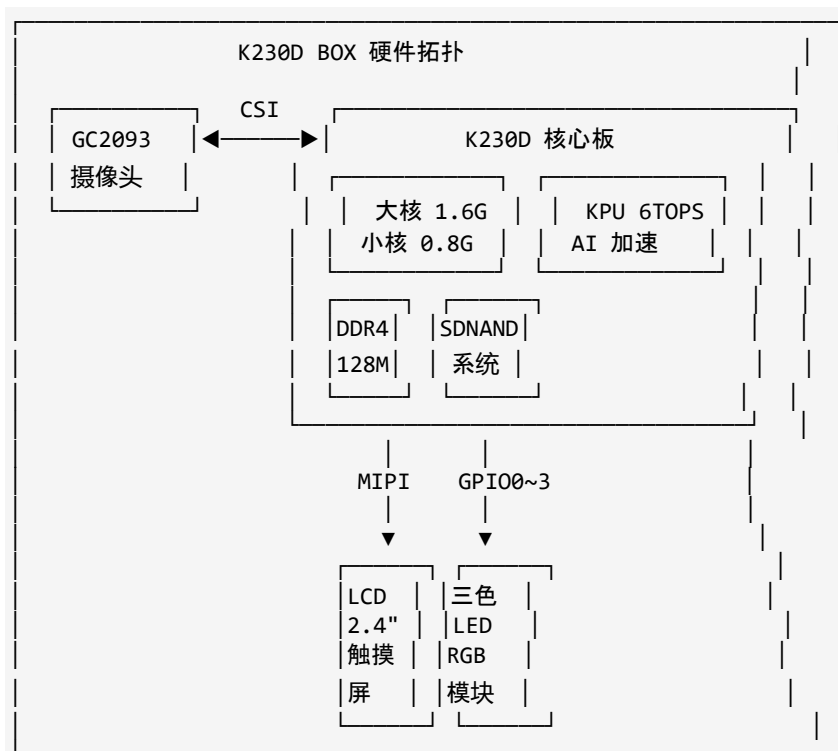
```

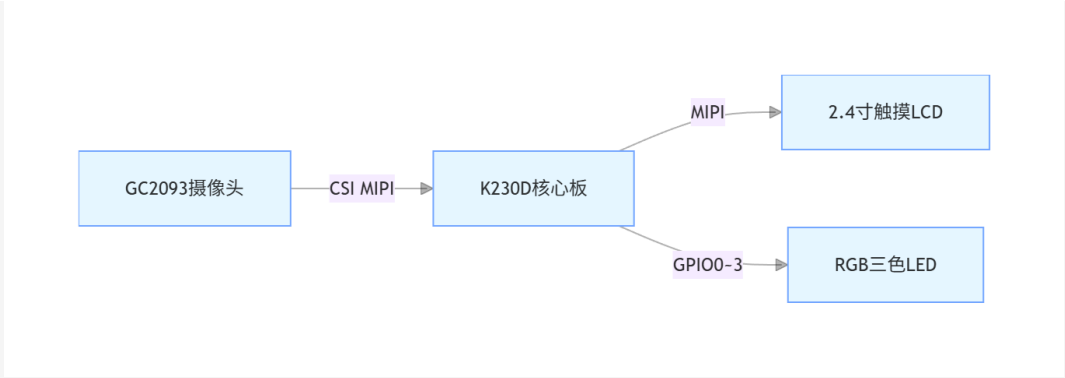
/sdcard/models/study_yolov8n_320.kmodel  # 模型
/sdcard/models/study_labels.txt          # 类别标签

```

三、硬件设计

3.1 系统框图



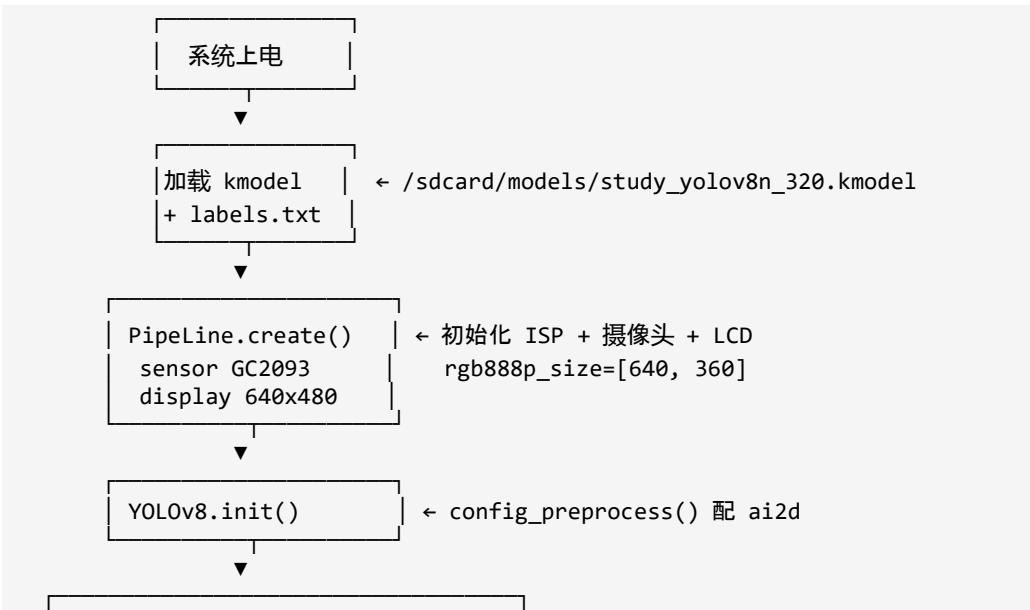


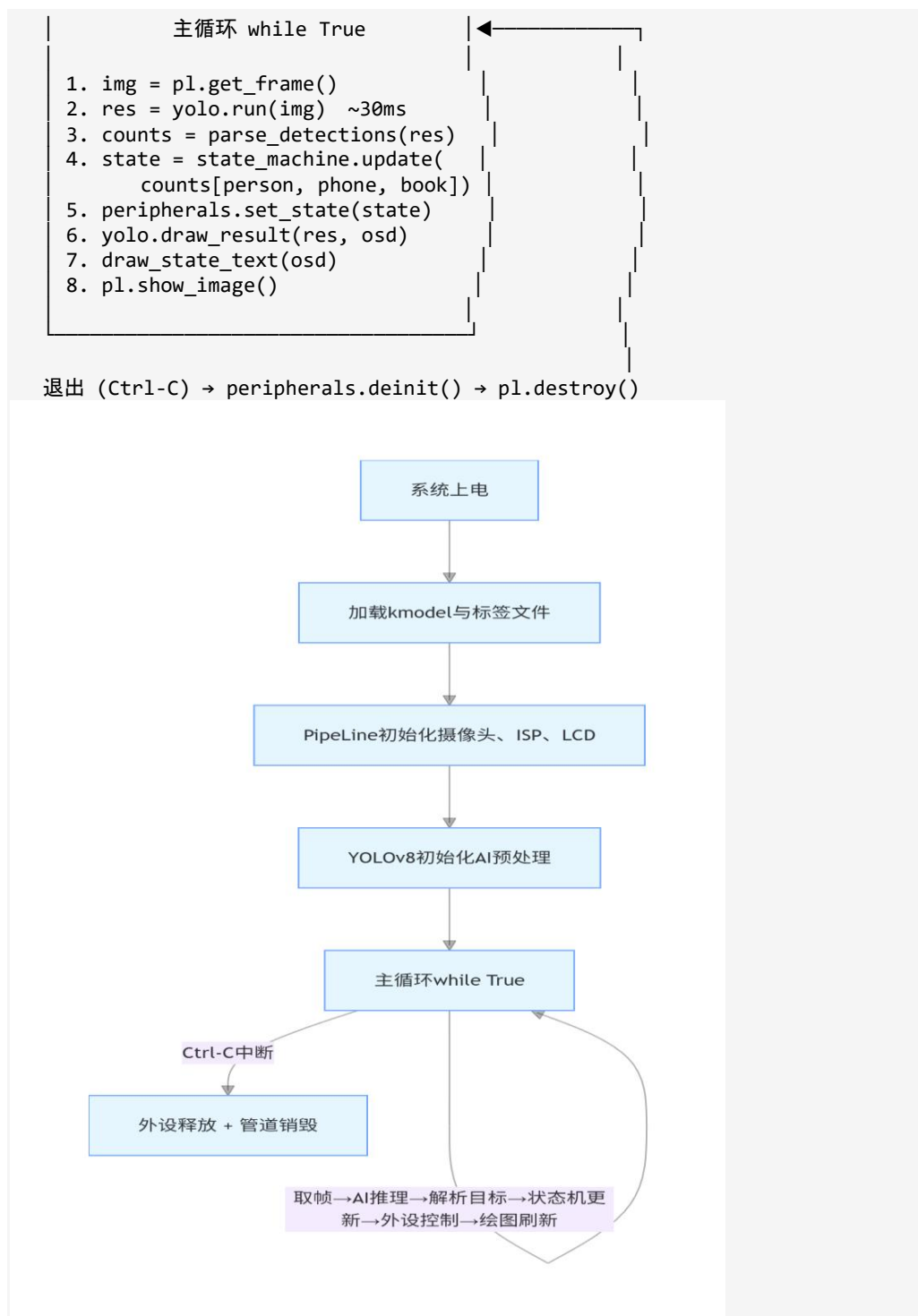
3.2 主要外设与引脚映射

外设	接口	引脚	说明
摄像头 GC2093	MIPI CSI1	—	板载直插，2MP 1920×1080
LCD 触摸屏	MIPI DSI	—	2.4 寸 640×480 + 触摸
三色 LED（蓝）	12Pin 排针 IO2	Pin(22)	检测到入
三色 LED（绿）	12Pin 排针 IO0	Pin(24)	无人 / 程序正常
三色 LED（红）	12Pin 排针 IO1	Pin(25)	检测到手机

四、边缘端软件设计

4.1 软件流程图





4.2 核心代码

4.2.1 设备主入口 `main.py`

```

from libs.PipeLine import Pipeline
from libs.Utils import *
import image, time
from config import RGB888P_SIZE, DISPLAY_MODE, DEBUG_MODE

```

```

from yolo_stage1 import YOLOStage1
from vision_state import StudyStateMachine, parse_detections
from peripherals import PeripheralsController
from utils import FPSCounter, InferTimer, debug_print

def main():
    pl = PipeLine(rgb888p_size=RGB888P_SIZE, display_mode=DISPLAY_MODE)
    pl.create()

    detector = YOLOStage1(pl.get_display_size())
    detector.init()

    state_machine = StudyStateMachine()
    peripherals = PeripheralsController()
    peripherals.init()

    fps = FPSCounter()
    timer = InferTimer()

    while True:
        img = pl.get_frame()
        if img is None:
            time.sleep_ms(10); continue

        timer.start()
        res = detector.run(img)
        infer_ms = timer.stop()
        fps.tick()

        counts = parse_detections(res)
        state = state_machine.update(
            counts["person"], counts["phone"],
            counts["book"], counts["laptop"]
        )

        peripherals.set_state(state)
        detector.draw_result(res, pl.osd_img)
        # draw_state_text + show_image 略

```

4.2.2 YOLOv8 检测结果解析 `vision\_state.py`

K230D CanMV 0.4.0 固件实测输出格式：[boxes_list, class_ids_list, scores_list]，三个独立列表。自动识别并兼容 A/B/C 三种历史格式：

```

def parse_detections(res):
    result = {"person": 0, "phone": 0, "book": 0, "laptop": 0}
    if res is None:
        return result

    # 实测格式 D: [boxes, class_ids, scores]
    if isinstance(res, (list, tuple)) and len(res) == 3:
        boxes, e1, e2 = res[0], res[1], res[2]
        if all(isinstance(x, (list, tuple)) for x in (boxes, e1, e2)):
            # 判断 e1 是 class_id (int), e2 是 score (0~1 float)
            if (all(isinstance(x, (int, float)) and x == int(x) for x in e1) and
                all(isinstance(x, float) and 0 <= x <= 1 for x in e2)):
                for i, box in enumerate(boxes):
                    cls_id = int(e1[i])
                    if 0 <= cls_id < len(LABELS):
                        result[LABELS[cls_id]] += 1
                return result
    # 兼容格式 A/B/C ... (省略)

```

```
return result
```

4.2.3 状态机 `vision\_state.py`

带时间窗口去抖的 4 状态机，状态优先级：PHONE > AWAY > FOCUS > PERSON：

```
class StudyStateMachine:
    def _classify_instant(self, person, phone, book, laptop):
        if phone > 0:
            return STATE_PHONE          # 任何时候看到手机都是分心
        if person == 0:
            return STATE_AWAY           # 无人
        if book > 0:
            return STATE_FOCUS          # 人在看书
        return STATE_PERSON            # 人在

    def update(self, person, phone, book, laptop):
        now = time.ticks_ms()
        instant = self._classify_instant(person, phone, book, laptop)
        if instant != self._candidate_state:
            self._candidate_state = instant
            self._candidate_start_ms = now
            return self._current_state
        if time.ticks_diff(now, self._candidate_start_ms) >=
self._required_ms(instant):
            self._current_state = instant
        return self._current_state
```

4.2.4 LED 外设 `peripherals.py`

K230D CanMV 固件使用 machine.Pin 类（不是 GPIO）：

```
from machine import Pin

class PeripheralsController:
    def init(self):
        self._pins = {
            'B': Pin(22, Pin.OUT),  # 蓝灯 - 检测到入
            'G': Pin(24, Pin.OUT),  # 绿灯 - 无人 / 程序运行
            'R': Pin(25, Pin.OUT),  # 红灯 - 检测到手机
        }

    def set_state(self, state):
        pattern = LED_PATTERNS.get(state, (0, 0, 0))
        b, g, r = pattern
        self._pins['B'].value(b)
        self._pins['G'].value(g)
        self._pins['R'].value(r)
```

LED_PATTERNS 查表（state → (蓝, 绿, 红)）：

状态	蓝	绿	红
AWAY / UNKNOWN	0	1	0
FOCUS / PERSON	1	0	0
PHONE	0	0	1

五、效果展示

5.1 AI 检测效果



场景	描述	检测结果	状态	LED
1. 桌前看书	person + book 在画面	person:1, book:1	FOCUS	蓝灯亮
2. 桌前看手机	person + phone	person:1, phone:1	PHONE	红灯亮
3. 只检测到人	person 单独	person:1	PERSON	蓝灯亮
4. 无人	镜头前无目标	全 0	AWAY	绿灯亮
5. 拿手机给镜头	person:0, phone:1	PHONE	红灯亮	

六、总结

6.1 完成情况

本项目完成了 K230D BOX 桌面学习状态识别系统的全链路开发：

1. 数据集准备 — 5 015 张图片预处理、标注修复、训练 / 验证集划分
2. 模型训练 — YOLOv8n 3 类目标检测，CPU 训练 100 epochs，最佳 mAP50 = 0.610
3. 模型转换 — ONNX 导出 (11.6 MB) → nncase INT8 量化 → KModel (3.2 MB)
4. 端侧部署 — 完整 5 个 .py 模块(config/main/yolo_stage1/vision_state / peripherals / utils)
5. 状态判定 — 带时间窗口去抖的 4 状态机 (FOCUS / PHONE / PERSON / AWAY)
6. 外设响应 — 3 色 LED (蓝/绿/红) 实时反馈学习状态
7. 实时性能 — 端侧 25 FPS，单帧 ~30 ms 推理，无需云端

6.2 遇到的问题与解决

问题	现象	原因	解决
sensor snapshot 失败	主循环 get_frame 报 `chn(2) failed(3)`	加载 YOLO 模型和 sensor 资源竞争	调整初始化顺序，加 warmup
GPIO 导入失败	`from machine import GPIO` 报 ImportError	K230D CanMV 用 `Pin` 类而非 `GPIO` 类	改用 `from machine import Pin`
灯不响应状态变化	状态机返回正确状态但灯不切换	YOLOv8 输出格式 `[boxes, scores, classes]` 与代码预期不一致	改 `parse_detections` 写自动识别 4 种格式
灯不亮且状态卡 AWAY	状态机对无人画面优先级过强	`if not has_person` 在最前	调整优先级：手机 > 无人 > 看书 > 只人
模型检测断续	弱检测 (0.28~0.5 置信度) 一帧有下一帧无	训练数据少 + 320 小输入	CONF_THRESH 降到 0.10，CONFIRM_SECONDS 缩到 0.3s
跨行 f-string 报错	peripherals.py 第 89 行 SyntaxError	CanMV MicroPython 不支持相邻 f-string 隐式拼接	改用 `+` 显式字符串拼接

6.3 收获

1. **理解了端侧 AI 部署的完整链路：**从数据采集到模型训练、量化、端侧推理，每个环节都有具体的工程难点，不是单纯“跑通模型”那么简单。
2. **熟悉了 K230D 工具链：**CanMV IDE + MicroPython + PipeLine + YOLO libs + nncase 转换工具，每个工具都有自己的 API 习惯和坑。
3. **掌握了嵌入式系统的状态机设计：**用时间窗口去抖避免状态抖动，优先级排序保证关键状态（手机）能最快响应。
4. **学会了工程化的调试方法：**串口打印分级、原始数据打印、状态机分支打点、硬件接线对照表，逐步缩小问题范围。
5. **解了“简单需求”背后的复杂性：**一个看似简单的“看手机亮红灯”功能，要解决 6 个工程问题才能真正稳定工作。